

CHAPTER 14

14 - 캐싱 + 스케줄링 (cache-aside · Redis · @Cron)

Node.js · Express · NestJS 강의 · 2026

상품 상세는 자주 읽히는데 매번 DB를 조회하면 낭비다. 이 장은 두 가지 실무 패턴을 더한다.

- **캐싱(cache-aside)** — 상품 상세·인기 목록을 캐시에 담아 DB 부하를 줄인다. 상품이 바뀌면 캐시를 버린다.
- **@Cron 스케줄링** — 일정 주기로 상품 통계를 집계해 적재하는 배치.

캐시는 두 단계로 간다. 먼저 NestJS 기본 캐시(프로세스 메모리)로 cache-aside를 구현해 개념을 익히고, 그 한계(재시작하면 증발, 서버 간 공유 불가)를 확인한 뒤 **설정 한 곳만 바꿔 Redis**로 옮긴다. 11장에서 SQLite → PostgreSQL로 교체했던 것과 같은 패턴이다 — **사용하는 코드는 그대로, 저장소만 교체**.

13장에 `Product.viewCount` (조회수)와 `DailyStat` (통계 스냅샷) 모델이 더해진다. 나머지(인증·관계)는 그대로다.

이 장에서 달라진 것 (← 13장 대비)

구분	파일 / 내용
신규	<code>src/stats/</code> (<code>@Cron</code> 통계 배치 모듈)
수정	<code>prisma/schema.prisma</code> — Product.viewCount · DailyStat 모델 · <code>app.module</code> — <code>CacheModule</code> · <code>ScheduleModule</code> 등록(캐시 저장소: 메모리 → Redis) · <code>products.service</code> — cache-aside (<code>findOne</code> / <code>popular</code>) + 쓰기 후 무효화 · <code>products.controller</code> — <code>popular</code> · <code>view</code> 추가
의존성	<code>+@nestjs/cache-manager</code> <code>+cache-manager</code> <code>+@nestjs/schedule</code> · (Redis 전환 시) <code>+keyv</code> <code>+@keyv/redis</code>
엔드포인트	<code>+GET /products/popular</code> <code>+POST /products/:id/view</code> <code>+GET /stats</code> <code>+POST /stats/run</code>

0. 사전 준비 — PostgreSQL

PostgreSQL이 떠 있어야 한다(11장에서 접속한 그 서버). Redis는 아직 필요 없다 — 이 장 중간(6번)에 설치한다.

1. 패키지 설치

```
npm i @nestjs/cache-manager@3 cache-manager@7
npm i @nestjs/schedule@6
```

@nestjs/cache-manager 3.x는 **Keyv** 기반이다. 저장소를 안 꽂으면 프로세스 메모리에 캐시한다 — 그래서 Redis 없이 먼저 시작할 수 있다.

2. 모듈 생성

```
nest g module stats
nest g service stats --no-spec
nest g controller stats --no-spec
```

3. 스키마 — viewCount · DailyStat + 마이그레이션

```
model Product {
  // ... 기존 필드 그대로 ...
  viewCount  Int          @default(0) // 14장에서 추가 - 조회수
}

// @Cron이 하루 한 번(데모에선 1분마다) 적재하는 스냅샷.
// day를 @unique로 두어 같은 날은 한 줄만 갱신(upsert)한다.
model DailyStat {
  id          Int          @id @default(autoincrement())
  day         String       @unique // "2026-06-09"
  productCount Int
  totalStock  Int
  createdAt   DateTime @default(now())
}
```

```
npx prisma migrate dev --name add-cache-stats
```

4. 코드 작성 순서 — 1단계는 메모리 캐시

순 서	파일	내용
1	src/app.module.ts	CacheModule.register({ isGlobal: true, ttl: 30_000 }) — 저장소 설정이 없으니 메모리 · ScheduleModule.forRoot()
2	src/products/products.service.ts	cache-aside — findOne / popular 는 캐시 먼저 · 쓰기 (update / remove / incrementView) 후 무효화
3	src/products/products.controller.ts	GET /products/popular · POST /products/:id/view 추가
4	src/stats/stats.service.ts	@Cron(CronExpression.EVERY_MINUTE) — DailyStat upsert
5	src/stats/stats.controller.ts	GET /stats · POST /stats/run (즉시 1회 실행)

각 파일의 코드와 설명은 해설서 14장에 있다. ttl 단위는 **밀리초**다(30_000 = 30초).

5. 실행 + 캐싱 확인 — 그리고 메모리 캐시의 한계

```
npm run start:dev
```

콘솔에 1분마다 [cron] 스냅샷 저장 ... 로그가 찍히면 스케줄러가 도는 것이다.

```

BASE=http://localhost:3000
login() { curl -s -X POST $BASE/auth/login -H "Content-Type: application/json" \
  -d '{"email":"'${1}',"password":"'${2}'" | jq -r .access_token; }
ADMIN=$(login admin@example.com admin123)

# 상품 등록(관리자도 판매자 라우트를 통과한다) + 조회수 올리기
curl -s -X POST $BASE/products -H "Authorization: Bearer $ADMIN" -H "Content-Type: application/json" \
  -d '{"name":"무선 이어폰","description":"x","price":89000,"stock":10,"categoryIds":[1]}'
curl -s -X POST $BASE/products/1/view
curl -s -X POST $BASE/products/1/view

# 인기 상품 - 처음엔 source:"db", 30초 안에 다시 부르면 source:"cache"
curl -s $BASE/products/popular
curl -s $BASE/products/popular      # ← cache

# 상세도 같은 패턴 (product:1 캐시)
curl -s $BASE/products/1            # db → 캐시 적재
curl -s $BASE/products/1            # cache hit
curl -s -X POST $BASE/products/1/view # 조회수 변경 → 캐시 무효화
curl -s $BASE/products/1            # 다시 db (최신 반영)

```

이제 한계를 확인하자. 서버를 재시작하고(`start:dev` 면 파일 저장만 해도 된다) 30초 안에 다시 불러 보라.

```
curl -s $BASE/products/popular      # 재시작 직후 → 다시 "db"
```

캐시가 프로세스 메모리에 있어서 서버와 함께 사라진 것이다. 데이터는 DB에 있으니 잃은 건 없지만, 재시작·배포 때마다 캐시가 텅 비고, 서버를 여러 대 띄우면 서로 캐시를 공유하지 못한다. 그래서 캐시를 별도 서버로 뺀다 — Redis다.

6. Redis로 갈아타기 — 설정 한 곳만 바꾼다

6-1. Redis 설치 (Homebrew)

```

brew install redis
brew services start redis      # 백그라운드로 시작 — 맥을 재시작해도 자동으로 켜진다
redis-cli ping                 # PONG 이 나오면 준비 끝

```

6-2. 어댑터 설치 + .env

```
npm i keyv @keyv/redis
```

`.env` 에 Redis 접속을 추가한다.

```
REDIS_HOST=localhost
REDIS_PORT=6379
```

6-3. app.module.ts — stores 한 곳만 교체

```
import { createKeyv } from "@keyv/redis";

CacheModule.registerAsync({
  isGlobal: true,
  useFactory: async () => ({
    // Keyv의 Redis 어댑터를 쏴준다. stores를 빼면 기본 저장소(프로세스 메모리)로 동작한다.
    stores: [
      createKeyv(
        `redis://${process.env.REDIS_HOST ?? "localhost"}:${process.env.REDIS_PORT ?? 6379}`,
      ),
    ],
    ttl: 30_000, // 기본 TTL(ms) - set()에서 개별 지정한 값이 우선
  }),
}),
```

products.service.ts 의 **cache-aside** 코드는 한 글자도 안 바뀐다. `cache.get/set/del` 을 부르는 쪽은 저장소가 메모리인지 Redis인지 모른다.

6-4. 확인 — 이번엔 재시작해도 살아 있다

```
npm run start:dev
```

```
curl -s $BASE/products/popular # db → Redis에 적재
curl -s $BASE/products/popular # cache

# Redis에 실제로 들어갔는지 직접 확인
redis-cli KEYS '*' # products:popular 가 보인다
```

서버를 재시작하고 30초 안에 다시 불러 보라 — 이번엔 **"cache"**가 유지된다. 캐시가 앱 프로세스 밖(Redis)에 있기 때문이다. 서버 여러 대가 같은 Redis를 보면 캐시도 공유된다.

7. 스케줄링 동작 확인

```
curl -s -X POST $BASE/stats/run # 1분 기다리지 않고 즉시 한 번 집계
curl -s $BASE/stats # 적재된 스냅샷 목록
```

핵심 개념

캐시 어사이드(cache-aside)

읽기: 캐시 확인 → 있으면 반환(hit) / 없으면 DB 조회 후 캐시에 저장(miss)
 쓰기: DB 변경 → 관련 캐시 삭제(del) ← "stale 캐시"를 막는 핵심

- `popular()` / `findOne()` 이 읽기, `incrementView()` / `update()` / `remove()` 가 쓰기 후 무효화.
- 전역 `CacheModule` (`app.module.ts`)이 저장소를 연결 → `@Inject(CACHE_MANAGER)` 로 주입.
- 상품 상세는 `product:<id>`, 인기 목록은 `products:popular` 키를 쓴다. 상품이 바뀌면 둘 다 버린다.

메모리 캐시 vs Redis — 언제 무엇

	메모리(기본)	Redis
설치	없음	Redis 서버 필요
속도	가장 빠름(같은 프로세스)	빠름(네트워크 한 홉)
재시작	캐시 증발	유지
서버 여러 대	각자 따로(불일치 위험)	공유

개발·단일 프로세스면 메모리로 충분하고, 운영에서 서버를 여러 대 띄우는 순간 Redis가 필요해진다. 전환 비용은 설정 한 곳이다 — 이것이 추상화(`cache-manager` + Keyv)의 값어치다.

@Cron 스케줄링

- `ScheduleModule.forRoot()` 등록 후 `@Cron(...)` 으로 메서드를 주기 실행.
- 데모는 `EVERY_MINUTE` . 실무는 `EVERY_DAY_AT_MIDNIGHT` 또는 크론식 `"0 0 * * *"` .
- `DailyStat.day` 를 `@unique` 로 두어 같은 날은 한 줄을 갱신(upsert)한다.

핵심 파일

파일	역할
app.module.ts	CacheModule (메모리 → Redis는 stores 설정만) + ScheduleModule.forRoot()
products/products.service.ts	cache-aside 읽기 + 쓰기 후 무효화 — 저장소와 무관
stats/stats.service.ts	@Cron 으로 통계 스냅샷 적재

따라잡기 — 완성본으로 실행 (보조 경로)

수업의 기본 경로는 위처럼 직접 만드는 것이다. 따라오다 놓쳤다면, 수업 저장소의 14/ 폴더(완성 코드, Redis 전환이 끝난 상태)로 이 장의 끝에서 다시 시작할 수 있다.

```
cd 14
npm install
cp .env.example .env          # 사용자·비밀번호를 본인 PostgreSQL 값으로 수정
npx prisma migrate dev --name init  # 이 폴더 전용 DB(nest_shop_14) - 자동 생성이 안 되면
DBeaver로 먼저 만든다
npm run seed
npm run start:dev
```

다음 장

- 15 - 장바구니·주문: 구매자가 장바구니에 담고 주문한다. 재고 차감·주문 생성을 트랜잭션으로 묶는다.